PYTEST — THE PRACTICAL REFERENCE

Methods, fixtures, commands, patterns and runnable examples

Python Testing Handbook

November 2026

Pytest: Complete Practical Guide

For: Python developers, automation testers, learners, Django/FastAPI developers and technical assessors.
Scope: Common and advanced *public* Pytest APIs and workflow, with examples. Pytest has a larger plugin and internal-hook ecosystem; this is a comprehensive practical guide rather than an inventory of every internal function.
Convention: Examples use Python 3.11+ and modern Pytest 9.x unless otherwise noted.

Reading map

Level	Recommended chapters	Goal
Beginner	1–5	Write and run reliable tests
Intermediate	6–13	Reuse setup, vary inputs, isolate dependencies
API/backend	14–18	Verify APIs, async code and data workflows
Advanced	19–25	Configure, extend, optimize and debug a suite

1. Install Pytest and start a project

Pytest discovers Python tests, runs assertions, manages reusable setup with fixtures, and reports detailed failures.

```
python -m venv .venv
# Linux/macOS
source .venv/bin/activate
# Windows PowerShell
# .venv\Scripts\Activate.ps1
python -m pip install -U pytest
python -m pytest --version
```

The basic pattern: Arrange inputs, Act by calling the function, Assert its output.

```
# calculator.py
def add(a, b):
    return a + b

# test_calculator.py
from calculator import add

def test_add_positive():
    assert add(2, 3) == 5

def test_add_negative():
    assert add(-4, -1) == -5

python -m pytest -v
```

Use `python -m pytest` if your environment has multiple Python installations: it runs Pytest using the selected interpreter.

2. Test discovery and project layout

By default, Pytest looks for `test_*.py` and `*_test.py` files, functions named `test_*`, and methods named `test_*` inside classes named `Test*` (without custom constructors). Keep tests independent; do not rely on execution order.

```
project/
├── app/
│   ├── __init__.py
│   └── calculator.py
├── tests/
│   ├── conftest.py
│   ├── test_calculator.py
│   └── test_api.py
├── pyproject.toml
└── requirements-dev.txt
```

```python
# tests/test_group.py
class TestArithmetic:
    def test_subtraction(self):
        assert 7 - 2 == 5

    def test_multiplication(self):
        assert 4 * 3 == 12
```

Useful identifiers: tests/test_group.py selects a file; tests/test_group.py::TestArithmetic selects a class; tests/test_group.py::TestArithmetic::test_subtraction selects one method.

3. Essential command-line methods and options

Command	What it does
pytest	Run collected tests
python -m pytest	Run with the current Python interpreter
pytest -v / -q	Verbose / quiet output
pytest -s	Disable output capture, show print
pytest -x	Stop after first failure
pytest --maxfail=3	Stop after 3 failures
pytest -k "login and not slow"	Select by test-name expression
pytest -m smoke	Select tests marked smoke
pytest --collect-only -q	List collected tests, do not run
pytest --lf / --ff	Run last failures / run failures first
pytest --tb=short	Short traceback
pytest --tb=no	Suppress failure tracebacks
pytest --durations=10	Show 10 slowest test durations
pytest -rA	Extra summary for all result categories
pytest --disable-warnings	Hide warning summary (not warnings themselves)
pytest --strict-markers	Error on unregistered marker names
pytest --fixtures	Show available fixtures
pytest --markers	Show registered markers
pytest --help	Full installed command-line option list
pytest --junitxml=report.xml	Create CI-friendly XML report
pytest --pdb	Open debugger on failure
pytest --setup-show	Show fixture setup/teardown

```
pytest tests/test_calculator.py -v
pytest tests/test_calculator.py::test_add_positive -q
pytest -k "add or subtraction" --tb=short
pytest -m "api and not slow"
```

Note: --cov, -n, --html and --timeout are not Pytest core options; they require separately installed plugins (see chapter 20).

4. Assertions: the most important operation

Pytest uses Python's ordinary assert, and enriches failure messages automatically.

```
def test_basic_assertions():
    assert 5 == 5
    assert 3 != 4
    assert 8 > 3
    assert 2 in [1, 2, 3]
    assert "py" in "pytest"
    assert {"a": 1} == {"a": 1}
    assert all(n > 0 for n in [2, 3, 4])
    assert any(n % 2 == 0 for n in [1, 3, 4])
```

Pattern	Appropriate use
assert actual == expected	Equality
assert actual != expected	Inequality
assert a > b	Ordering
assert item in collection	Membership
assert obj is None	Identity with None
assert isinstance(obj, dict)	Type/category
assert values	Truthy nonempty objects
assert result, "message"	Custom failure explanation

Caution: is checks identity, not value equality. assert a is b is usually incorrect for strings, lists and computed numbers.

Floating-point comparisons: pytest.approx

Floating-point calculations should use a tolerance rather than strict equality.

```
import pytest

def test_discount():
    total = 0.1 + 0.2
    assert total == pytest.approx(0.3)
    assert total == pytest.approx(0.3, rel=1e-6)
    assert total == pytest.approx(0.3, abs=1e-9)
```

rel is relative tolerance; abs is absolute tolerance. Use it for computed numerical results; not for exact currency amounts represented as Decimal.

5. Exception, warning and failure APIs

pytest.raises() — test expected exceptions

```
import pytest

def divide(a, b):
    if b == 0:
        raise ValueError("division by zero")
    return a / b

def test_divide_by_zero():
    with pytest.raises(ValueError, match="division by zero"):
        divide(5, 0)

def test_error_object():
    with pytest.raises(ValueError) as exc_info:
        divide(5, 0)
    assert "division by zero" in str(exc_info.value)
```

match is a *regular expression*. Use re.escape(...) for text with special regex characters. Keep the with block short so it catches only the intended operation.

pytest.warns() and pytest.deprecated_call()

```
import warnings
import pytest

def old_api():
    warnings.warn("old API", DeprecationWarning)

def test_warning():
    with pytest.warns(DeprecationWarning, match="old API"):
        old_api()

def test_deprecated_call():
    with pytest.deprecated_call():
        old_api()
```

pytest.fail(), pytest.skip(), pytest.xfail()

```
import pytest

def test_manual_failure():
    result = "ok"
    if result != "ok":
        pytest.fail("Unexpected operation result")

def test_runtime_skip():
    pytest.skip("External service unavailable in this environment")

def test_known_problem():
    pytest.xfail("Known issue #123 is not fixed yet")
```

pytest.fail() fails immediately; pytest.skip() records a skip; pytest.xfail() reports an expected failure at runtime. Never turn genuine failures into expected failures merely to make a suite green.

pytest.importorskip()

```
import pytest

numpy = pytest.importorskip("numpy")

def test_numpy_average():
    assert numpy.mean([1, 2, 3]) == 2
```

This skips the module's tests when the optional dependency is unavailable.

6. Parametrization: test many cases with one function

@pytest.mark.parametrize()

```
import pytest

@pytest.mark.parametrize(
    "a,b,expected",
    [(2, 3, 5), (-1, 4, 3), (0, 0, 0)],
)
def test_add_many(a, b, expected):
    assert a + b == expected

@pytest.mark.parametrize(
    "value,expected",
    [(" PYTHON ", "python"), (" PyTest ", "pytest")],
    ids=["python-normalization", "pytest-normalization"],
)
def test_clean_text(value, expected):
    assert value.strip().lower() == expected
```

Individual cases with pytest.param

```
@pytest.mark.parametrize(
    "numerator,denominator,expected",
    [
        pytest.param(8, 2, 4, id="regular"),
        pytest.param(9, 3, 3, marks=pytest.mark.smoke, id="smoke"),
    ],
)
def test_division(numerator, denominator, expected):
    assert numerator / denominator == expected
```

Stacked decorators produce a Cartesian product

```
@pytest.mark.parametrize("x", [1, 2])
@pytest.mark.parametrize("y", [10, 20])
def test_pairs(x, y):
    assert x + y in (11, 12, 21, 22)
# Four separate test cases are collected.
```

Best practice: Use descriptive ids when hundreds of cases exist. Keep case values small and readable; load datasets from fixtures if they are large.

7. Fixtures: setup, reuse and dependency injection

A fixture supplies known data or resources. A test requests it by adding the fixture's name as a parameter.

```
import pytest

@pytest.fixture
def sample_user():
    return {"id": 1, "name": "Asha", "active": True}

def test_user_name(sample_user):
    assert sample_user["name"] == "Asha"
```

Fixture dependencies

```
@pytest.fixture
def base_price():
    return 100

@pytest.fixture
def discounted_price(base_price):
    return base_price * 0.9

def test_discounted_price(discounted_price):
    assert discounted_price == pytest.approx(90)
```

Fixture scopes

Scope	Resource lifetime	Typical use
function	One test (default)	Mutable objects, DB transactions
class	One test class	Shared class context
module	One test module	Module-local integration setup
package	One package	Package-wide resources
session	Entire test run	Expensive immutable configurations

```python
@pytest.fixture(scope="session")
def service_config():
    return {"base_url": "https://example.invalid", "retries": 2}
```

Avoid shared mutable state in session fixtures; one test should not alter the next test's environment.

yield fixtures for reliable teardown

```python
@pytest.fixture
def temporary_connection():
    connection = {"open": True}
    yield connection
    connection["open"] = False  # teardown after test

def test_connection(temporary_connection):
    assert temporary_connection["open"] is True
```

Code after yield runs during fixture teardown, including after a test assertion fails.

8. Fixture variations: autouse, parameters, factories, request

autouse=True — automatically apply a fixture

```python
@pytest.fixture(autouse=True)
def clear_shared_list():
    SHARED.clear()

SHARED = []

def test_fresh_state():
    assert SHARED == []
```

Keep autouse use limited and obvious; implicit modifications can make test behavior hard to understand.

Parametrized fixtures and request.param

```python
@pytest.fixture(params=["sqlite", "postgres"], ids=["lite", "server"])
def backend_name(request):
    return request.param

def test_backend_is_supported(backend_name):
    assert backend_name in {"sqlite", "postgres"}
```

Factory fixtures — make multiple objects per test

```python
@pytest.fixture
def make_user():
    def factory(name="Guest", role="viewer"):
        return {"name": name, "role": role}
    return factory

def test_multiple_users(make_user):
    admin = make_user("Ravi", "admin")
    viewer = make_user("Mina")
    assert admin["role"] == "admin"
    assert viewer["role"] == "viewer"
```

request fixture — inspect the current test

```
@pytest.fixture
def test_name(request):
    return request.node.name

def test_request_example(test_name):
    assert "test_request_example" in test_name
```

Shared fixtures in conftest.py

```
# tests/conftest.py
import pytest

@pytest.fixture
def auth_headers():
    return {"Authorization": "Bearer example-test-token"}

# tests/test_headers.py
def test_headers(auth_headers):
    assert auth_headers["Authorization"].startswith("Bearer ")
```

Pytest loads accessible conftest.py fixtures automatically; there is no need to import the fixture manually.

9. Marks, skip rules and expected failures

Markers label tests and apply conditional behavior. Register custom names in pytest.ini or pyproject.toml to avoid warnings.

```
import sys
import pytest

@pytest.mark.smoke
def test_health():
    assert True

@pytest.mark.skip(reason="Feature not available yet")
def test_future_feature():
    assert False

@pytest.mark.skipif(sys.platform == "win32", reason="Unix-only")
def test_unix_feature():
    assert True

@pytest.mark.xfail(reason="Known bug", strict=True)
def test_known_bug():
    assert 1 == 2

pytest -m smoke -v
pytest -m "smoke or api" -v
pytest -m "not slow" -v
pytest -rsx
```

Skip vs xfail: Skip means a test is not executed; xfail runs the test and treats a failure as expected. With strict=True, an unexpected pass (XPASS) is treated as a failure so the team notices that the expectation may be outdated.

usefixtures and filterwarnings markers

```
@pytest.fixture
def environment_ready():
    assert True

@pytest.mark.usefixtures("environment_ready")
def test_requires_environment():
    assert 10 > 2

@pytest.mark.filterwarnings("ignore:old API:DeprecationWarning")
def test_legacy_behavior():
    assert True
```

filterwarnings should be targeted; broad warning suppression can hide real compatibility issues.

10. Built-in fixtures and their methods

Built-in fixture	Purpose / commonly used methods
tmp_path	Unique pathlib.Path directory for a test
tmp_path_factory	.mktemp(name) creates session-level temporary dirs
capsys	.readouterr(), .disabled() for Python-level stdout/stderr
capfd	.readouterr(), .disabled() for OS file-descriptor output
capsysbinary / capfdbinary	Capture output as bytes
caplog	.text, .records, .record_tuples, .at_level(...)
monkeypatch	.setattr, .delattr, .setitem, .delitem, .setenv, .delenv, .syspath_prepend, .chdir, .context
recwarn	Access warnings; .pop(WarningType)
request	.param, .node, .config, .getfixturevalue(...), .addfinalizer(...)
pytestconfig	.getoption(...), .getini(...)
cache	.get(key, default), .set(key, value)
doctest_namespace	Inject names for doctests
record_property	Store a per-test JUnit XML property
record_testsuite_property	Store a suite-level JUnit XML property
subtests	Run subchecks when supported by installed Pytest version
tmpdir / tmpdir_factory	Older py.path variants; prefer tmp_path

Run pytest --fixtures -v to inspect the actual fixtures provided by your installed version and plugins.

11. Temporary paths and file testing

```python
def save_report(path, text):
    path.write_text(text, encoding="utf-8")

def test_write_report(tmp_path):
    report = tmp_path / "report.txt"
    save_report(report, "completed")
    assert report.exists()
    assert report.read_text(encoding="utf-8") == "completed"
```

Shared temporary directory factory

```python
import pytest

@pytest.fixture(scope="session")
def sample_file(tmp_path_factory):
    folder = tmp_path_factory.mktemp("input-data")
    path = folder / "numbers.csv"
    path.write_text("value\n10\n20\n", encoding="utf-8")
    return path

def test_csv_data(sample_file):
    assert "20" in sample_file.read_text(encoding="utf-8")
```

Use tmp_path instead of writing outputs into a developer's repository or home directory.

12. Monkeypatch and mocking

monkeypatch — temporary change with automatic restoration

```
import os

def read_mode():
    return os.getenv("APP_MODE", "prod")

def test_environment(monkeypatch):
    monkeypatch.setenv("APP_MODE", "test")
    assert read_mode() == "test"

# service.py
def fetch_status():
    raise RuntimeError("network must not be called")

def healthy():
    return fetch_status() == "ok"

# test_service.py
import service

def test_healthy(monkeypatch):
    monkeypatch.setattr(service, "fetch_status", lambda: "ok")
    assert service.healthy()
```

Common operations:

```
monkeypatch.setenv("TOKEN", "abc")
monkeypatch.delenv("TOKEN", raising=False)
monkeypatch.setitem(settings, "DEBUG", True)
monkeypatch.delitem(settings, "OLD_KEY", raising=False)
monkeypatch.setattr(module, "function_name", fake_function)
monkeypatch.delattr(module, "old_attr", raising=False)
monkeypatch.chdir(tmp_path)
monkeypatch.syspath_prepend(str(tmp_path))
```

These are *illustrative statements inside a test*; settings, module, fake_function, and tmp_path refer to values you would define or request in that test.

Standard library unittest.mock.patch

```
# payment.py
from urllib.request import urlopen

def get_receipt_code():
    return "R-001"

# test_payment.py
from unittest.mock import patch
import payment

def test_receipt_code():
    with patch.object(payment, "get_receipt_code", return_value="R-999") as fake:
        assert payment.get_receipt_code() == "R-999"
        fake.assert_called_once_with()
```

Critical rule: Patch the symbol *where the code under test looks it up*, not necessarily where the function was originally defined.

pytest-mock plugin and mocker fixture

```
python -m pip install pytest-mock

def test_mock_call(mocker):
    fake = mocker.Mock(return_value=42)
    assert fake("input") == 42
    fake.assert_called_once_with("input")
```

mocker is a plugin fixture, not built into core Pytest.

13. Capturing output, logs and warnings

capsys: print() and Python-level output

```
def announce():
    print("Task completed")

def test_stdout(capsys):
    announce()
    captured = capsys.readouterr()
    assert "Task completed" in captured.out
    assert captured.err == ""
```

capfd: subprocess/native file-descriptor output

```
import os

def test_fd_output(capfd):
    os.write(1, b"ready\n")
    out, err = capfd.readouterr()
    assert "ready" in out
```

caplog: logging

```
import logging

def process():
    logging.getLogger("demo").warning("low balance")

def test_logging(caplog):
    with caplog.at_level(logging.WARNING, logger="demo"):
        process()
    assert "low balance" in caplog.text
    assert any(r.levelname == "WARNING" for r in caplog.records)
```

recwarn: inspect emitted warnings

```
import warnings

def test_warning_collection(recwarn):
    warnings.warn("please migrate", DeprecationWarning)
    warning = recwarn.pop(DeprecationWarning)
    assert "migrate" in str(warning.message)
```

14. Test classes and grouping

Pytest supports class-based organization without using unittest.TestCase.

```
class TestAccount:
    def test_initial_balance(self):
        balance = 100
        assert balance == 100

    def test_deposit(self):
        balance = 100
        balance += 40
        assert balance == 140
```

Avoid __init__ in Pytest test classes. Each test method should obtain its own necessary state from fixtures instead of relying on a previous method's changes.

15. Testing dictionaries, lists and pandas data

Collections and deterministic ordering

```
def test_list_result():
    result = ["sql", "python", "pytest"]
    assert len(result) == 3
    assert "python" in result
    assert result == ["sql", "python", "pytest"]

def test_unordered_result():
    assert set(["b", "a"]) == {"a", "b"}
```

Pandas DataFrame test (optional dependency)

```
python -m pip install pandas

import pandas as pd
from pandas.testing import assert_frame_equal

def test_clean_dataframe():
    actual = pd.DataFrame({"name": ["A", "B"], "score": [8, 9]})
    expected = pd.DataFrame({"name": ["A", "B"], "score": [8, 9]})
    assert_frame_equal(actual, expected)
```

For numerical ML predictions, use tolerances. For Pandas frames, use pandas.testing.assert_frame_equal to obtain diagnostics on dtypes, indices and values.

16. API testing with FastAPI and TestClient

Install backend testing dependencies:

```
python -m pip install fastapi httpx pytest
```

Application:

```
# app.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/items", status_code=201)
def create_item(item: Item):
    if item.price < 0:
        raise HTTPException(status_code=400, detail="invalid price")
    return item.model_dump()
```

Tests:

```
# test_api.py
from fastapi.testclient import TestClient
from app import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}

def test_create_item():
    response = client.post("/items", json={"name": "Book", "price": 250})
    assert response.status_code == 201
    assert response.json()["name"] == "Book"
```

```
def test_invalid_price():
    response = client.post("/items", json={"name": "Book", "price": -1})
    assert response.status_code == 400
    assert response.json()["detail"] == "invalid price"

def test_missing_name_validation():
    response = client.post("/items", json={"price": 20})
    assert response.status_code == 422
```

TestClient is a synchronous interface suitable for ordinary FastAPI route tests. Use a fresh test database or controlled dependency override for endpoints that write persistent data.

17. FastAPI dependency overrides and authentication tests

```
# auth_app.py
from fastapi import Depends, FastAPI

app = FastAPI()

def current_user():
    return {"role": "guest"}

@app.get("/me")
def me(user=Depends(current_user)):
    return user

# test_auth_app.py
from fastapi.testclient import TestClient
from auth_app import app, current_user


def test_current_user_override():
    app.dependency_overrides[current_user] = lambda: {"role": "admin"}
    try:
        with TestClient(app) as client:
            response = client.get("/me")
        assert response.status_code == 200
        assert response.json() == {"role": "admin"}
    finally:
        app.dependency_overrides.clear()
```

JWT testing checklist: Verify missing token (401 or your app's documented response), malformed token, expired token, valid token, insufficient role (403), and a successful authorized request. Never embed production signing secrets in tests; use a dedicated test secret and isolated configuration.

18. Asynchronous tests: pytest-asyncio

```
python -m pip install pytest-asyncio

# test_async.py
import asyncio
import pytest

async def calculate_async(value):
    await asyncio.sleep(0)
    return value * 2

@pytest.mark.asyncio
async def test_async_function():
    assert await calculate_async(5) == 10
```

pytest-asyncio supports strict (explicitly marked) and auto modes. In strict mode, async fixtures should use @pytest_asyncio.fixture.

```python
# test_async_fixture.py
import pytest
import pytest_asyncio

@pytest_asyncio.fixture
async def async_value():
    return 12

@pytest.mark.asyncio
async def test_async_fixture(async_value):
    assert async_value == 12
```

For async HTTP requests you can use HTTPX AsyncClient with ASGITransport, but ensure application lifespan/startup resources are handled when tests depend on them.

19. Configuration: pyproject.toml and pytest.ini

Recommended **pyproject.toml** configuration:

```toml
[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = ["test_*.py", "*_test.py"]
addopts = "-ra --strict-markers"
markers = [
  "smoke: fast critical checks",
  "api: HTTP endpoint tests",
  "slow: tests that take extra time",
]
filterwarnings = [
  "error::DeprecationWarning:myproject.*",
]
```

Alternative **pytest.ini** configuration:

```ini
[pytest]
testpaths = tests
addopts = -ra --strict-markers
markers =
    smoke: fast critical checks
    api: HTTP endpoint tests
    slow: tests that take extra time
```

Choose one main configuration file to avoid confusion. Set strict markers so misspellings like @pytest.mark.smoek are caught.

20. Useful ecosystem plugins (install separately)

Plugin	Install	Typical command / feature
pytest-cov	pip install pytest-cov	pytest --cov=app --cov-report=term-missing
pytest-xdist	pip install pytest-xdist	pytest -n auto (parallel workers)
pytest-html	pip install pytest-html	pytest --html=report.html --self-contained-html
pytest-mock	pip install pytest-mock	mocker fixture
pytest-asyncio	pip install pytest-asyncio	@pytest.mark.asyncio
pytest-timeout	pip install pytest-timeout	pytest --timeout=10

```
python -m pip install pytest-cov pytest-xdist pytest-html
pytest --cov=app --cov-report=term-missing --cov-report=html
pytest -n auto
pytest --html=test-report.html --self-contained-html
```

Coverage interpretation: Coverage measures executed lines/branches, not whether the test assertions are meaningful. Also consider boundary cases, failure modes and integration paths.

Parallel safety: `-n auto` may run tests concurrently in different processes. Avoid global file names, shared DB state, and order dependencies.

21. Advanced hooks, custom options and pytestconfig

Hooks let a project extend collection and reporting. Put project-local hooks in `conftest.py` or package them as a plugin.

```
# conftest.py
def pytest_addoption(parser):
    parser.addoption(
        "--environment",
        action="store",
        default="test",
        help="Target environment name",
    )

import pytest

@pytest.fixture
def environment(pytestconfig):
    return pytestconfig.getoption("environment")

# test_environment.py
def test_environment(environment):
    assert environment in {"test", "staging"}

pytest --environment=staging -v
```

Collection customization example

```
# conftest.py
def pytest_collection_modifyitems(items):
    # Useful for inspection or adding markers programmatically.
    for item in items:
        if "integration" in item.nodeid:
            item.add_marker("slow")
```

Register any dynamically added marker name. Hook signatures and supported pluggy behavior may evolve; verify advanced hook implementations against the current official API reference.

22. Running selective tests and debugging failures

```
pytest tests/test_users.py
pytest tests/test_users.py::test_create_user
pytest -k "login and not expired"
pytest -m smoke
pytest --lf
pytest --ff
pytest -x --pdb
pytest --setup-show -v
pytest --collect-only -q
pytest --durations=5
pytest --trace
```

Useful choices: `-s` prints captured output live; `--pdb` starts Python debugging after a failure; `--setup-show` reveals fixture order. `pytest -k` uses test-name expressions, whereas `-m` uses registered marker expressions.

23. Tests for errors, edge cases and security boundaries

A useful test suite should verify more than the happy path. Consider:

- **Typical input:** Normal positive, expected output.
- **Boundaries:** Zero, empty string, empty array, maximum accepted length.
- **Invalid input:** Incorrect type, out-of-range value, missing required value.

- **Exceptions:** Expected failure class and meaningful message.
- **State:** New state, existing state, duplicate requests, idempotent operation.
- **Access:** Unauthenticated user, wrong role, authenticated authorized user.
- **External failures:** Timeout, dependency error and fallback behavior.

```
import pytest

def validate_age(age):
    if not isinstance(age, int) or isinstance(age, bool):
        raise TypeError("age must be an integer")
    if not 0 <= age <= 120:
        raise ValueError("age out of range")
    return age

@pytest.mark.parametrize("age", [0, 18, 120])
def test_valid_ages(age):
    assert validate_age(age) == age

@pytest.mark.parametrize("age", [-1, 121])
def test_invalid_range(age):
    with pytest.raises(ValueError):
        validate_age(age)

@pytest.mark.parametrize("age", [None, 18.5, "18", True])
def test_invalid_types(age):
    with pytest.raises(TypeError):
        validate_age(age)
```

24. Complete runnable mini-project

This compact project demonstrates application code, fixtures, parametrization and exception tests together.

```
pytest-starter/
├── shop.py
├── conftest.py
├── test_shop.py
└── pytest.ini
```

shop.py

```
class Cart:
    def __init__(self):
        self.items = []

    def add(self, price):
        if price < 0:
            raise ValueError("price cannot be negative")
        self.items.append(price)

    def total(self):
        return sum(self.items)
```

conftest.py

```
import pytest
from shop import Cart

@pytest.fixture
def cart():
    return Cart()
```

test_shop.py

```
import pytest

def test_empty_cart(cart):
    assert cart.total() == 0

@pytest.mark.parametrize("prices,expected", [
    ([100, 50], 150),
    ([20], 20),
    ([0, 0], 0),
])
```

```
def test_cart_total(cart, prices, expected):
    for price in prices:
        cart.add(price)
    assert cart.total() == expected

def test_negative_price(cart):
    with pytest.raises(ValueError, match="negative"):
        cart.add(-10)
```

pytest.ini

```
[pytest]
addopts = -ra -v

cd pytest-starter
python -m pip install pytest
python -m pytest
# Expected: 5 passing test cases
```

25. Best practices: stable, maintainable tests

1. **One behavior per test:** A failing test should identify one broken expectation.
2. **Descriptive names:** test_register_rejects_duplicate_email beats test_01.
3. **Arrange-Act-Assert:** Keep setup, behavior and verification easy to scan.
4. **Isolate state:** Use fixtures, fake services and separate databases/files.
5. **Prefer standard assert:** Do not build your own heavyweight assertion wrappers.
6. **Do not depend on test order:** Each test must run successfully by itself.
7. **Avoid real external networks:** Mock or explicitly separate integration tests.
8. **Test failures too:** Validation errors and permission denials matter.
9. **Keep fixtures focused:** Hide repetitive setup, not test logic.
10. **Keep CI deterministic:** Pin dependencies, manage time zones, control randomness.

26. Common problems and how to fix them

Symptom	Likely cause	Fix
collected 0 items	File/function naming mismatch	Use test_*.py and test_* functions
ModuleNotFoundError	Wrong environment/import layout	Activate venv; python -m pytest; install package editable if appropriate
fixture 'x' not found	Fixture not in scope or missing plugin	Check conftest.py, imports, installed plugins
PytestUnknownMarkWarning	Custom mark unregistered	Register in config, use --strict-markers
async def functions are not natively supported	Async plugin absent/configured incorrectly	Install/configure pytest-asyncio
Test passes alone, fails in suite	Leaking shared state	Reset with fixtures; isolate resources
assert 0.300000... == 0.3	Float rounding	Use pytest.approx
Tests hang on HTTP requests	Real network / timeout missing	Mock network or use explicit timeouts
--cov not recognized	pytest-cov missing	Install pytest-cov in same environment
-n not recognized	pytest-xdist missing	Install pytest-xdist
--html not recognized	pytest-html missing	Install pytest-html
Teardown missing after fixture	Used return instead of yield	Place cleanup after yield

27. Compact API and method quick-reference

API / feature	Syntax or example	Why use it
pytest.fixture	@pytest.fixture(scope="function")	Reusable setup
pytest.mark.parametrize	@pytest.mark.parametrize("x", [1, 2])	Data-driven tests
pytest.param	pytest.param(1, id="one")	Per-case ID/mark
pytest.mark.skip	@pytest.mark.skip(reason="...")	Always skip
pytest.mark.skipif	@pytest.mark.skipif(flag, reason="...")	Conditional skip
pytest.mark.xfail	@pytest.mark.xfail(strict=True)	Expected failure
pytest.mark.usefixtures	@pytest.mark.usefixtures("db")	Activate fixture
pytest.mark.filterwarnings	@pytest.mark.filterwarnings("ignore:...")	Targeted filter
pytest.approx	assert x == pytest.approx(y)	Float comparison
pytest.raises	with pytest.raises(ValueError):	Exception check
pytest.warns	with pytest.warns(UserWarning):	Warning check
pytest.deprecated_call	with pytest.deprecated_call():	Deprecated behavior
pytest.fail	pytest.fail("reason")	Immediate failure
pytest.skip	pytest.skip("reason")	Runtime skip
pytest.xfail	pytest.xfail("reason")	Runtime xfail
pytest.importorskip	np = pytest.importorskip("numpy")	Optional dependency
request.getfixturevalue	request.getfixturevalue("name")	Dynamic fixture lookup
request.addfinalizer	request.addfinalizer(cleanup)	Register cleanup
monkeypatch.setattr	monkeypatch.setattr(obj, "x", fake)	Patch attribute
monkeypatch.setenv	monkeypatch.setenv("MODE", "test")	Patch environment
capsys.readouterr	captured = capsys.readouterr()	Inspect output
caplog.at_level	with caplog.at_level(logging.INFO):	Capture logging
tmp_path	tmp_path / "data.csv"	Isolated files
cache.get / .set	cache.get("ns/key", None)	Cached run state
pytestconfig.getoption	pytestconfig.getoption("-v")	Read CLI option
pytest.main	pytest.main(["-q", "tests"])	Programmatic invocation

pytest.main() is mainly useful in scripts or tooling. A normal command-line invocation is preferable when running a project test suite.

28. Recommended learning exercises

1. Write five parametrized tests for an add, subtract, multiply, divide module; test division by zero.
2. Build a fixture that creates sample customer dictionaries; test duplicate email handling.
3. Use tmp_path to create and verify a CSV report.
4. Use monkeypatch to replace a payment gateway call so tests never hit a real gateway.
5. Add one FastAPI GET, one POST, one invalid-payload test and one JWT authorization test.
6. Register a smoke marker; select it with pytest -m smoke; generate coverage using pytest-cov.

29. Further reading / authoritative documentation

- Core introduction: https://docs.pytest.org/en/stable/getting-started.html
- Public API and core command options: https://docs.pytest.org/en/stable/reference/reference.html
- Built-in fixture catalog: https://docs.pytest.org/en/stable/reference/fixtures.html
- Fixture lifetime and teardown: https://docs.pytest.org/en/stable/how-to/fixtures.html
- Parameterization: https://docs.pytest.org/en/stable/how-to/parametrize.html
- Markers: https://docs.pytest.org/en/stable/how-to/mark.html
- pytest-asyncio: https://pytest-asyncio.readthedocs.io/en/stable/
- FastAPI testing: https://fastapi.tiangolo.com/tutorial/testing/
- pytest-cov: https://pytest-cov.readthedocs.io/en/latest/

End of guide. Use `pytest --help` and `pytest --fixtures` to inspect the exact options and fixtures available in your local environment, including installed plugins.